

React + TypeScript Frontend Lesson Plan - Part 1

Question Canvas: Vite SPA + Enterprise API Client + Canvas Workspace

Baseline: React + TypeScript / Vite / Visual Studio Code / August 2026

Student outcome: follow every step in Visual Studio Code and finish with a working, testable project.

Lesson map

1. What the Part 1 frontend will do
 2. Prerequisites and backend assumptions
 3. Create the Vite React TypeScript application
 4. Define the API types and HTTP error contract
 5. Build enterprise authentication state
 6. Build the project/question workspace data hook
 7. Build forms and accessible components
 8. Build the blank canvas workspace
 9. Assemble the single-screen application
 10. Style the screen
 11. Run the full stack and verify behavior
 12. React hook review and Part 2 image plan
- A. File tree
 - B. Troubleshooting

1. What the Part 1 frontend will do

This PDF starts from the empty frontend folder created by the backend lesson. You will build a single-screen React/TypeScript SPA that authenticates against the ASP.NET Core API, creates projects, saves questions, displays saved data, and reserves a real HTML canvas for Part 2 image rendering and manipulation.

1.1 Single-screen design

The application uses one route and conditionally renders either an authentication card or the authenticated workspace. After login, the screen contains a project panel, a question panel, and an image workspace. Separate React components are still used because component boundaries are an implementation concern, not separate pages.

1.2 Canvas is the right Part 2 component

Keep the canvas. An `<img>` element is excellent for displaying a server image, but Part 2 requires sharpening, noise removal, and other pixel operations. The best flow is to load the returned image, draw it to a canvas, read `ImageData`, apply filters, and redraw. Part 1 therefore creates and correctly sizes the canvas but leaves its pixels blank.

1.3 Hooks used naturally in this lesson

Hook	Where it belongs
<code>useState</code>	Auth form fields, filter text, local UI flags.
<code>useEffect</code>	Session bootstrap, project loading, canvas sizing/cleanup.
<code>useContext</code>	Make authentication state/actions available to the app.
<code>useReducer</code>	Coordinate multi-field workspace state and async transitions.
<code>useCallback</code>	Expose stable async actions from context/custom hooks.
<code>useMemo</code>	Memoize auth context value and filtered project list.
<code>useRef</code>	Hold the canvas DOM element and ignore stale project-detail responses.
<code>useId</code>	Accessible label/input IDs inside reusable forms.
<code>useDeferredValue</code>	Keep project filtering responsive if the project list grows.

Do not force hooks: Enterprise React is not about using every hook. Hooks such as `useLayoutEffect`, `useImperativeHandle`, `useSyncExternalStore`, or `useOptimistic` should appear only when the problem calls for them. This lesson uses many hooks because they fit the actual workflow.

2. Prerequisites and backend assumptions

- Complete the .NET Part 1 PDF first.
- The API runs at <https://localhost:7043>.
- The API has `/api/auth` Identity endpoints plus `/api/projects` endpoints.
- Node.js satisfies the current Vite requirement (Node 20.19+ or 22.12+ is acceptable; Node 22 LTS is a good choice).
- npm is available in the VS Code terminal.

Verify Node and npm

```
node --version
npm --version
```

Start the backend in one terminal before the final integration section:

From EnterpriseQuestionCanvas

```
cd backend
dotnet run --project src/QuestionCanvas.Api/QuestionCanvas.Api.csproj
```

Checkpoint: Do not continue until every item below is true.

[] Backend health/live returns Healthy at <https://localhost:7043/health/live>.

[] node --version satisfies Vite requirements.

3. Create the Vite React TypeScript application

Open a second VS Code terminal at EnterpriseQuestionCanvas and scaffold directly into the existing frontend directory.

Create the app

```
cd frontend
npm create vite@latest . -- --template react-ts
npm install
```

Vite may ask whether to continue because the folder exists. It should be empty. Continue only if frontend contains no files you need to keep.

3.1 Configure the development reverse proxy

Browser requests will use relative /api URLs. Vite proxies those requests to ASP.NET Core. This makes the browser see same-origin requests during development, simplifies cookie handling, and avoids hard-coding the API base URL throughout components.

frontend/vite.config.ts

```
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'

export default defineConfig({
  plugins: [react()],
  server: {
    port: 5173,
    proxy: {
      '/api': {
        target: 'https://localhost:7043',
        changeOrigin: true,
        secure: false,
      },
    },
  },
})
```

secure: false: This applies only to the local Vite proxy so it can accept the ASP.NET Core development certificate. It is not a production TLS setting. Production should terminate valid HTTPS and should not disable certificate verification.

3.2 Remove starter UI files

- Delete src/assets/react.svg.
- Replace src/App.tsx and src/App.css later in this lesson.
- Replace src/index.css later in this lesson.

Confirm the untouched scaffold builds before continuing

```
npm run build
```

4. Define the API types and HTTP error contract

Do not scatter fetch calls and response parsing through UI components. Start with a small typed API boundary that knows about HTTP, JSON, cookie credentials, and ProblemDetails.

4.1 Create shared API types

frontend/src/types/api.ts

```
export type ProjectSummary = {
  id: string
  projectName: string
}
```

```

    questionCount: number
    createdUtc: string
  }

export type QuestionItem = {
  id: string
  question: string
  createdUtc: string
}

export type ProjectDetails = {
  id: string
  projectName: string
  createdUtc: string
  questions: QuestionItem[]
}

export type SessionInfo = {
  email: string | null
}

export type ProblemDetails = {
  type?: string
  title?: string
  status?: number
  detail?: string
  instance?: string
  traceId?: string
  errors?: Record<string, string[]>
}

```

4.2 Create a reusable API error and fetch wrapper

frontend/src/api/http.ts

```

import type { ProblemDetails } from '../types/api'

export class ApiError extends Error {
  constructor(
    public readonly status: number,
    public readonly problem?: ProblemDetails,
  ) {
    super(problem?.detail ?? problem?.title ?? `Request failed with ${status}`)
    this.name = 'ApiError'
  }
}

export async function apiFetch<T>(
  input: string,
  init: RequestInit = {},
): Promise<T> {
  const headers = new Headers(init.headers)

  if (init.body && !headers.has('Content-Type')) {
    headers.set('Content-Type', 'application/json')
  }

  const response = await fetch(input, {
    ...init,
    headers,
    credentials: 'include',
  })

  if (!response.ok) {
    let problem: ProblemDetails | undefined

    if (response.headers.get('content-type')?.includes('json')) {
      problem = (await response.json()) as ProblemDetails
    }

    throw new ApiError(response.status, problem)
  }

  if (response.status === 204) {
    return undefined as T
  }

  const text = await response.text()

```

```

    if (!text) {
      return undefined as T
    }

    return JSON.parse(text) as T
  }

```

credentials: include is required for the Identity cookie to participate in fetch requests. This wrapper also creates one place to add correlation headers, telemetry, retry policy for safe operations, or API-version headers later.

5. Build enterprise authentication state

Authentication is cross-cutting application state, so it belongs in a context with a narrow custom hook. Components should ask `auth.login()` or `auth.logout()`; they should not know Identity endpoint details.

5.1 Create the AuthContext

`frontend/src/auth/AuthContext.tsx`

```

import {
  createContext,
  useCallback,
  useEffect,
  useMemo,
  useState,
  type PropsWithChildren,
} from 'react'
import { ApiError, apiFetch } from '../api/http'
import type { SessionInfo } from '../types/api'

type AuthStatus = 'loading' | 'anonymous' | 'authenticated'

type AuthContextValue = {
  status: AuthStatus
  email: string | null
  login: (email: string, password: string) => Promise<void>
  register: (email: string, password: string) => Promise<void>
  logout: () => Promise<void>
}

export const AuthContext = createContext<AuthContextValue | null>(null)

export function AuthProvider({ children }: PropsWithChildren) {
  const [status, setStatus] = useState<AuthStatus>('loading')
  const [email, setEmail] = useState<string | null>(null)

  const refreshSession = useCallback(async () => {
    try {
      const session = await apiFetch<SessionInfo>('/api/auth/session')
      setEmail(session.email)
      setStatus('authenticated')
    } catch (error) {
      if (error instanceof ApiError && error.status === 401) {
        setEmail(null)
        setStatus('anonymous')
        return
      }
      throw error
    }
  }, [])

  useEffect(() => {
    void refreshSession().catch((error) => {
      console.error('Unable to restore the session.', error)
      setEmail(null)
      setStatus('anonymous')
    })
  }, [refreshSession])

  const login = useCallback(async (loginEmail: string, password: string) => {
    setStatus('loading')

    try {
      await apiFetch<void>('/api/auth/login?useCookies=true', {

```

```

        method: 'POST',
        body: JSON.stringify({ email: loginEmail, password }),
    })
    await refreshSession()
  } catch (error) {
    setStatus('anonymous')
    throw error
  }
}, [refreshSession])

const register = useCallback(async (
  registerEmail: string,
  password: string,
) => {
  await apiFetch<void>('/api/auth/register', {
    method: 'POST',
    body: JSON.stringify({ email: registerEmail, password }),
  })

  await login(registerEmail, password)
}, [login])

const logout = useCallback(async () => {
  await apiFetch<void>('/api/auth/logout', { method: 'POST' })
  setEmail(null)
  setStatus('anonymous')
}, [])

const value = useMemo<AuthContextValue>(() => ({
  status,
  email,
  login,
  register,
  logout,
}), [status, email, login, register, logout])

return <AuthContext.Provider value={value}>{children}</AuthContext.Provider>
}

```

5.2 Create the useAuth hook

frontend/src/auth/useAuth.ts

```

import { useContext } from 'react'
import { AuthContext } from '../AuthContext'

export function useAuth() {
  const value = useContext(AuthContext)

  if (!value) {
    throw new Error('useAuth must be used inside AuthProvider.')
  }

  return value
}

```

StrictMode note: React StrictMode can run development effects more than once to expose unsafe side effects. Session bootstrap is a GET and is safe to repeat. Do not put project creation or other mutations in a mount effect.

6. Build the project/question workspace data hook

The workspace has related async state: project list, selected project, loading state, and errors. A reducer makes those transitions explicit. The custom hook also owns all project API calls so presentation components stay focused.

6.1 Create useWorkspace

frontend/src/hooks/useWorkspace.ts

```

import { useCallback, useEffect, useReducer, useRef } from 'react'
import { apiFetch } from '../api/http'
import type {

```

```

ProjectDetails,
ProjectSummary,
QuestionItem,
} from '../types/api'

type WorkspaceState = {
  projects: ProjectSummary[]
  selected: ProjectDetails | null
  loadingProjects: boolean
  loadingDetails: boolean
  error: string | null
}

type Action =
  | { type: 'reset' }
  | { type: 'projectsLoading' }
  | { type: 'projectsLoaded'; projects: ProjectSummary[] }
  | { type: 'detailsLoading' }
  | { type: 'detailsLoaded'; project: ProjectDetails | null }
  | { type: 'failed'; message: string }

const initialState: WorkspaceState = {
  projects: [],
  selected: null,
  loadingProjects: false,
  loadingDetails: false,
  error: null,
}

function reducer(state: WorkspaceState, action: Action): WorkspaceState {
  switch (action.type) {
    case 'reset':
      return initialState
    case 'projectsLoading':
      return { ...state, loadingProjects: true, error: null }
    case 'projectsLoaded':
      return { ...state, projects: action.projects, loadingProjects: false }
    case 'detailsLoading':
      return { ...state, loadingDetails: true, error: null }
    case 'detailsLoaded':
      return { ...state, selected: action.project, loadingDetails: false }
    case 'failed':
      return {
        ...state,
        loadingProjects: false,
        loadingDetails: false,
        error: action.message,
      }
  }
}

export function useWorkspace(enabled: boolean) {
  const [state, dispatch] = useReducer(reducer, initialState)
  const detailsRequestId = useRef(0)

  const loadProjects = useCallback(async (signal?: AbortSignal) => {
    dispatch({ type: 'projectsLoading' })

    try {
      const projects = await apiFetch<ProjectSummary[]>('/api/projects', {
        signal,
      })
      dispatch({ type: 'projectsLoaded', projects })
    } catch (error) {
      if (error instanceof DOMException && error.name === 'AbortError') return
      dispatch({
        type: 'failed',
        message: error instanceof Error ? error.message : 'Unable to load projects.',
      })
    }
  }, [])

  const selectProject = useCallback(async (projectId: string) => {
    const requestId = ++detailsRequestId.current
    dispatch({ type: 'detailsLoading' })

    try {
      const project = await apiFetch<ProjectDetails>(
        `/api/projects/${projectId}`,
      )
    }
  }, [])

```



```

    if (requestId === detailsRequestId.current) {
      dispatch({ type: 'detailsLoaded', project })
    }
  } catch (error) {
    if (requestId === detailsRequestId.current) {
      dispatch({
        type: 'failed',
        message: error instanceof Error ? error.message : 'Unable to load project.',
      })
    }
  }
}, [])

const createProject = useCallback(async (projectName: string) => {
  const created = await apiFetch<ProjectSummary>('/api/projects', {
    method: 'POST',
    body: JSON.stringify({ projectName }),
  })

  await loadProjects()
  await selectProject(created.id)
  return created
}, [loadProjects, selectProject])

const addQuestion = useCallback(async (
  projectId: string,
  question: string,
): Promise<QuestionItem> => {
  const created = await apiFetch<QuestionItem>(
    `/api/projects/${projectId}/questions`,
    {
      method: 'POST',
      body: JSON.stringify({ question }),
    },
  )

  await loadProjects()
  await selectProject(projectId)
  return created
}, [loadProjects, selectProject])

useEffect(() => {
  if (!enabled) {
    dispatch({ type: 'reset' })
    return
  }

  const controller = new AbortController()
  void loadProjects(controller.signal)

  return () => controller.abort()
}, [enabled, loadProjects])

return {
  ...state,
  loadProjects,
  selectProject,
  createProject,
  addQuestion,
}
}

```

`detailsRequestId` is an enterprise-grade race guard. If a user clicks project A and immediately clicks project B, a slow response for A must not overwrite the newer B selection.

7. Build forms and accessible components

7.1 Authentication panel

`frontend/src/components/AuthPanel.tsx`

```

import { useId, useState, type FormEvent } from 'react'
import { useAuth } from '../auth/useAuth'

type Mode = 'login' | 'register'

export function AuthPanel() {

```

```

const auth = useAuth()
const emailId = useId()
const passwordId = useId()
const [mode, setMode] = useState<Mode>('login')
const [email, setEmail] = useState('')
const [password, setPassword] = useState('')
const [error, setError] = useState<string | null>(null)

async function onSubmit(event: FormEvent<HTMLFormElement>) {
  event.preventDefault()
  setError(null)

  try {
    if (mode === 'login') {
      await auth.login(email, password)
    } else {
      await auth.register(email, password)
    }
  } catch (err) {
    setError(err instanceof Error ? err.message : 'Authentication failed.')
  }
}

return (
  <section className="auth-card" aria-labelledby="auth-title">
    <h1 id="auth-title">Question Canvas</h1>
    <p>Sign in to access your private projects and questions.</p>

    <div className="segmented" role="group" aria-label="Authentication mode">
      <button type="button" onClick={() => setMode('login')} aria-pressed={mode === 'login'}>
        Sign in
      </button>
      <button type="button" onClick={() => setMode('register')} aria-pressed={mode === 'register'}>
        Register
      </button>
    </div>

    <form onSubmit={onSubmit}>
      <label htmlFor={emailId}>Email</label>
      <input
        id={emailId}
        type="email"
        autoComplete="email"
        value={email}
        onChange={(event) => setEmail(event.target.value)}
        required
      />

      <label htmlFor={passwordId}>Password</label>
      <input
        id={passwordId}
        type="password"
        autoComplete={mode === 'login' ? 'current-password' : 'new-password'}
        value={password}
        onChange={(event) => setPassword(event.target.value)}
        minLength={12}
        required
      />

      {mode === 'register' && (
        <p className="hint">Use 12+ characters with upper/lowercase, a number, and a symbol.</p>
      )}

      {error && <p className="error" role="alert">{error}</p>}

      <button type="submit" disabled={auth.status === 'loading'}>
        {auth.status === 'loading'
          ? 'Working...'
          : mode === 'login' ? 'Sign in' : 'Create account'}
      </button>
    </form>
  </section>
)
}

```

7.2 Project creation form

frontend/src/components/ProjectForm.tsx

```

import { useId, useState, type FormEvent } from 'react'

export function ProjectForm({
  onCreate,
}: {
  onCreate: (projectName: string) => Promise<void>
}) {
  const inputId = useId()
  const [projectName, setProjectName] = useState('')
  const [busy, setBusy] = useState(false)
  const [error, setError] = useState<string | null>(null)

  async function onSubmit(event: FormEvent<HTMLFormElement>) {
    event.preventDefault()
    setBusy(true)
    setError(null)

    try {
      await onCreate(projectName)
      setProjectName('')
    } catch (err) {
      setError(err instanceof Error ? err.message : 'Unable to create project.')
    } finally {
      setBusy(false)
    }
  }

  return (
    <form onSubmit={onSubmit} className="stack-form">
      <label htmlFor={inputId}>Project Name</label>
      <input
        id={inputId}
        value={projectName}
        onChange={(event) => setProjectName(event.target.value)}
        maxLength={120}
        required
      />
      {error && <p className="error" role="alert">{error}</p>}
      <button type="submit" disabled={busy}>
        {busy ? 'Saving...' : 'Create project'}
      </button>
    </form>
  )
}

```

7.3 Question form

frontend/src/components/QuestionForm.tsx

```

import { useId, useState, type FormEvent } from 'react'

export function QuestionForm({
  disabled,
  onCreate,
}: {
  disabled: boolean
  onCreate: (question: string) => Promise<void>
}) {
  const inputId = useId()
  const [question, setQuestion] = useState('')
  const [busy, setBusy] = useState(false)
  const [error, setError] = useState<string | null>(null)

  async function onSubmit(event: FormEvent<HTMLFormElement>) {
    event.preventDefault()
    setBusy(true)
    setError(null)

    try {
      await onCreate(question)
      setQuestion('')
    } catch (err) {
      setError(err instanceof Error ? err.message : 'Unable to save question.')
    } finally {
      setBusy(false)
    }
  }
}

```

```

return (
  <form onSubmit={onSubmit} className="stack-form">
    <label htmlFor={inputId}>Question</label>
    <textarea
      id={inputId}
      value={question}
      onChange={(event) => setQuestion(event.target.value)}
      maxLength={2000}
      rows={5}
      disabled={disabled}
      required
    />
    <div className="form-meta">{question.length}/2000</div>
    {error && <p className="error" role="alert">{error}</p>}
    <button type="submit" disabled={disabled || busy}>
      {busy ? 'Saving...' : 'Save question'}
    </button>
  </form>
)
}

```

8. Build the blank canvas workspace

The canvas itself remains visually blank in Part 1. The component only keeps its backing pixel buffer synchronized with its CSS size and device pixel ratio. That prevents a blurry canvas when Part 2 begins drawing images.

frontend/src/components/ImageWorkspace.tsx

```

import { useEffect, useRef } from 'react'

export function ImageWorkspace() {
  const canvasRef = useRef<HTMLCanvasElement>(null)

  useEffect(() => {
    const canvas = canvasRef.current
    if (!canvas) return

    const resizeCanvas = () => {
      const rect = canvas.getBoundingClientRect()
      const dpr = window.devicePixelRatio || 1

      canvas.width = Math.max(1, Math.round(rect.width * dpr))
      canvas.height = Math.max(1, Math.round(rect.height * dpr))

      const context = canvas.getContext('2d')
      if (!context) return

      context.setTransform(dpr, 0, 0, dpr, 0, 0)
      context.clearRect(0, 0, rect.width, rect.height)
    }

    resizeCanvas()

    const observer = new ResizeObserver(resizeCanvas)
    observer.observe(canvas)

    return () => observer.disconnect()
  }, [])

  return (
    <section className="panel image-panel" aria-labelledby="image-title">
      <div className="panel-heading">
        <div>
          <h2 id="image-title">Answer Image</h2>
          <p>Part 2 will render the generated answer here.</p>
        </div>
        <span className="badge">Part 2</span>
      </div>

      <canvas
        ref={canvasRef}
        className="answer-canvas"
        aria-label="Generated answer image workspace"
      />

      <fieldset className="future-controls" disabled>
        <legend>Image controls - enabled in Part 2</legend>
      </fieldset>
    </section>
  )
}

```

```

        <label>
          Sharpen
          <input type="range" min="0" max="100" defaultValue="0" />
        </label>
        <label>
          Noise reduction
          <input type="range" min="0" max="100" defaultValue="0" />
        </label>
        <button type="button">Reset image</button>
      </fieldset>
    </section>
  )
}

```

The disabled controls deliberately preview the Part 2 interaction without pretending the algorithms exist. In Part 2 their values can move into a useReducer-based image-processing state, while the canvas ref remains unchanged.

9. Assemble the single-screen application

9.1 Add an application-level error boundary

API errors are expected operational errors and are handled near the form/hook. Render-time component failures are different. A React error boundary prevents a broken render tree from leaving the screen blank with no recovery message.

frontend/src/components/AppErrorBoundary.tsx

```

import { Component, type ErrorInfo, type PropsWithChildren } from 'react'

type State = { hasError: boolean }

export class AppErrorBoundary extends Component<PropsWithChildren, State> {
  state: State = { hasError: false }

  static getDerivedStateFromError(): State {
    return { hasError: true }
  }

  componentDidCatch(error: Error, info: ErrorInfo) {
    console.error('Unhandled React render error', error, info)
  }

  render() {
    if (this.state.hasError) {
      return (
        <main className="fatal-error">
          <h1>Question Canvas could not render.</h1>
          <p>Refresh the page. If the problem continues, check the browser console.</p>
        </main>
      )
    }

    return this.props.children
  }
}

```

9.2 Replace App.tsx

frontend/src/App.tsx

```

import {
  useCallback,
  useDeferredValue,
  useMemo,
  useState,
} from 'react'
import './App.css'
import { useAuth } from './auth/useAuth'
import { AuthPanel } from './components/AuthPanel'
import { ImageWorkspace } from './components/ImageWorkspace'
import { ProjectForm } from './components/ProjectForm'
import { QuestionForm } from './components/QuestionForm'
import { useWorkspace } from './hooks/useWorkspace'

```

```

export default function App() {
  const auth = useAuth()
  const workspace = useWorkspace(auth.status === 'authenticated')
  const [projectFilter, setProjectFilter] = useState('')
  const deferredFilter = useDeferredValue(projectFilter)

  const filteredProjects = useMemo(() => {
    const needle = deferredFilter.trim().toLocaleLowerCase()
    if (!needle) return workspace.projects

    return workspace.projects.filter((project) =>
      project.projectName.toLocaleLowerCase().includes(needle),
    )
  }, [deferredFilter, workspace.projects])

  const createProject = useCallback(async (projectName: string) => {
    await workspace.createProject(projectName)
  }, [workspace])

  const createQuestion = useCallback(async (question: string) => {
    if (!workspace.selected) return
    await workspace.addQuestion(workspace.selected.id, question)
  }, [workspace])

  if (auth.status === 'loading') {
    return <main className="centered-status">Checking session...</main>
  }

  if (auth.status === 'anonymous') {
    return (
      <main className="auth-shell">
        <AuthPanel />
      </main>
    )
  }

  return (
    <main className="app-shell">
      <header className="app-header">
        <div>
          <h1>Question Canvas</h1>
          <p>Projects and questions are private to {auth.email}</p>
        </div>
        <button type="button" className="secondary" onClick={() => void auth.logout()}>
          Sign out
        </button>
      </header>

      {workspace.error && (
        <div className="error-banner" role="alert">{workspace.error}</div>
      )}

      <div className="workspace-grid">
        <section className="panel projects-panel" aria-labelledby="projects-title">
          <div className="panel-heading">
            <div>
              <h2 id="projects-title">Projects</h2>
              <p>Each project groups multiple questions.</p>
            </div>
          </div>

          <ProjectForm onCreate={createProject} />

          <label htmlFor="project-filter">Filter projects</label>
          <input
            id="project-filter"
            value={projectFilter}
            onChange={(event) => setProjectFilter(event.target.value)}
            placeholder="Type a project name"
          />

          <div className="project-list" aria-busy={workspace.loadingProjects}>
            {filteredProjects.map((project) => (
              <button
                type="button"
                key={project.id}
                className={workspace.selected?.id === project.id ? 'project selected' : 'project'}
                onClick={() => void workspace.selectProject(project.id)}
              >
                <strong>{project.projectName}</strong>
                <span>{project.questionCount} question(s)</span>
              </button>
            ))}
          </div>
        </section>
      </div>
    </main>
  )
}

```

```

        </button>
      )}}
      {!workspace.loadingProjects && filteredProjects.length === 0 && (
        <p className="empty">No matching projects.</p>
      )}
    </div>
  </section>

  <section className="panel questions-panel" aria-labelledby="questions-title">
    <div className="panel-heading">
      <div>
        <h2 id="questions-title">
          {workspace.selected?.projectName ?? 'Questions'}
        </h2>
        <p>Select a project, then save one or more questions.</p>
      </div>
    </div>

    <QuestionForm
      disabled={!workspace.selected || workspace.loadingDetails}
      onCreate={createQuestion}
    />

    <div className="question-list" aria-busy={workspace.loadingDetails}>
      {workspace.selected?.questions.map((item) => (
        <article key={item.id} className="question-card">
          <p>{item.question}</p>
          <time dateTime={item.createdUtc}>
            {new Date(item.createdUtc).toLocaleString()}
          </time>
        </article>
      ))}

      {workspace.selected && workspace.selected.questions.length === 0 && (
        <p className="empty">This project has no questions yet.</p>
      )}
    </div>
  </section>

  <ImageWorkspace />
</div>
</main>
)
}

```

Callback dependency note: App passes the whole workspace object into two useCallback dependency lists. This is readable for the lesson, but because useWorkspace returns a new object each render those callbacks are not strongly memoized. In a performance-sensitive component tree, destructure the stable action functions first and depend on those exact functions. Correctness is unaffected here.

9.3 Replace main.tsx

frontend/src/main.tsx

```

import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import './index.css'
import App from './App'
import { AuthProvider } from './auth/AuthContext'
import { AppErrorBoundary } from './components/AppErrorBoundary'

createRoot(document.getElementById('root')!).render(
  <StrictMode>
    <AppErrorBoundary>
      <AuthProvider>
        <App />
      </AuthProvider>
    </AppErrorBoundary>
  </StrictMode>,
)

```

10. Style the screen

The CSS is intentionally plain application UI rather than a component library. Part 2 can introduce a design system without changing data/auth architecture.

10.1 Replace index.css

frontend/src/index.css

```
:root {
  font-family: Inter, ui-sans-serif, system-ui, -apple-system, BlinkMacSystemFont, "Segoe UI", sans-serif;
  color: #18222d;
  background: #f3f6f8;
  font-synthesis: none;
  text-rendering: optimizeLegibility;
}

* { box-sizing: border-box; }

body {
  margin: 0;
  min-width: 320px;
  min-height: 100vh;
}

button, input, textarea { font: inherit; }
button { cursor: pointer; }
button:disabled { cursor: not-allowed; opacity: 0.6; }

input, textarea {
  width: 100%;
  border: 1px solid #b8c5cf;
  border-radius: 8px;
  padding: 0.65rem 0.75rem;
  background: white;
}

input:focus, textarea:focus, button:focus-visible {
  outline: 3px solid rgba(31, 78, 120, 0.25);
  outline-offset: 2px;
}

label { font-weight: 650; }
```

10.2 Replace App.css

frontend/src/App.css

```
.app-shell { max-width: 1500px; margin: 0 auto; padding: 1.25rem; }
.app-header { display: flex; justify-content: space-between; gap: 1rem; align-items: center; margin-bottom: 1rem; }
.app-header h1, .panel h2 { margin: 0; }
.app-header p, .panel-heading p { margin: 0.25rem 0 0; color: #667481; }

.workspace-grid {
  display: grid;
  grid-template-columns: minmax(250px, 0.8fr) minmax(320px, 1fr) minmax(420px, 1.5fr);
  gap: 1rem;
  align-items: start;
}

.panel, .auth-card {
  background: white;
  border: 1px solid #d9e1e7;
  border-radius: 14px;
  box-shadow: 0 8px 24px rgba(31, 52, 69, 0.06);
  padding: 1rem;
}

.panel-heading { display: flex; justify-content: space-between; gap: 1rem; align-items: start; margin-bottom: 1rem; }
.stack-form { display: grid; gap: 0.55rem; margin-bottom: 1rem; }
.stack-form button, .auth-card button { border: 0; border-radius: 8px; padding: 0.7rem 0.9rem; background: #1f4e78; color: white; font-weight: 700; }
.secondary { border: 1px solid #9aabb8; border-radius: 8px; padding: 0.65rem 0.9rem; background: white; color: #243746; }
```



```

.project-list, .question-list { display: grid; gap: 0.6rem; margin-top: 0.8rem; max-height: 440px; overflow:
auto; }
.project { text-align: left; display: grid; gap: 0.2rem; border: 1px solid #d5dfe6; border-radius: 9px;
padding: 0.75rem; background: #fff; color: inherit; }
.project.selected { border-color: #1f4e78; background: #eef5fa; }
.project span, .question-card time { color: #697987; font-size: 0.85rem; }
.question-card { border: 1px solid #e1e7eb; border-radius: 9px; padding: 0.75rem; }
.question-card p { margin: 0 0 0.5rem; white-space: pre-wrap; }

.answer-canvas { width: 100%; height: 430px; display: block; border: 1px dashed #a9b8c3; border-radius: 10px;
background: #fff; }
.future-controls { display: grid; gap: 0.6rem; margin-top: 1rem; }
.future-controls label { display: grid; grid-template-columns: 120px 1fr; gap: 0.6rem; align-items: center; }
.badge { border-radius: 999px; padding: 0.25rem 0.55rem; background: #edf1f4; color: #52616d; font-size:
0.8rem; font-weight: 700; }

.auth-shell, .centered-status, .fatal-error { min-height: 100vh; display: grid; place-items: center; padding:
1rem; }
.auth-card { width: min(430px, 100%); }
.auth-card form { display: grid; gap: 0.55rem; }
.segmented { display: grid; grid-template-columns: 1fr 1fr; gap: 0.4rem; margin: 1rem 0; }
.segmented button[aria-pressed="false"] { background: #e8eef2; color: #263746; }
.hint, .form-meta, .empty { color: #697987; font-size: 0.85rem; }
.form-meta { text-align: right; }
.error, .error-banner { color: #8f1d1d; }
.error-banner { background: #fff0f0; border: 1px solid #f0b6b6; border-radius: 8px; padding: 0.75rem; margin-
bottom: 1rem; }

@media (max-width: 1050px) {
  .workspace-grid { grid-template-columns: 1fr 1fr; }
  .image-panel { grid-column: 1 / -1; }
}

@media (max-width: 720px) {
  .workspace-grid { grid-template-columns: 1fr; }
  .image-panel { grid-column: auto; }
  .app-header { align-items: flex-start; flex-direction: column; }
  .answer-canvas { height: 330px; }
}

```

Checkpoint: Do not continue until every item below is true.

- [] npm run build succeeds.
- [] No TypeScript errors appear in VS Code Problems.
- [] The canvas has a responsive blank drawing surface and disabled Part 2 controls.

11. Run the full stack and verify behavior

11.1 Start both development servers

Terminal 1 - repository root

```

cd backend
dotnet run --project src/QuestionCanvas.Api/QuestionCanvas.Api.csproj

```

Terminal 2 - repository root

```

cd frontend
npm run dev

```

Open the Vite URL, normally <http://localhost:5173>. The browser should not directly navigate to the API port for normal SPA use.

11.2 Complete the end-to-end checklist

1. Register Alice with a password that satisfies the backend policy.
2. After registration, confirm the app automatically signs Alice in.
3. Create a project named Bird Study.

- Create two questions under Bird Study.
- Refresh the browser. The auth cookie should restore the session and the project list should reload.
- Select Bird Study and confirm both persisted questions return.
- Create a second project and use the filter box to demonstrate useDeferredValue/useMemo filtering.
- Sign out. Confirm the workspace disappears and the authentication card returns.
- Sign in again and confirm persisted data returns.
- Verify the Answer Image canvas is empty and its controls are disabled.

11.3 Use browser DevTools as an enterprise debugging tool

- Network: verify requests go to /api/... on the Vite origin and are proxied to ASP.NET Core.
- Network: inspect 401/400/409 ProblemDetails payloads when deliberately causing errors.
- Application/Storage: verify an HttpOnly auth cookie exists. Do not try to read it from React code.
- Console: there should be no uncaught application errors.
- Accessibility: tab through forms and verify visible focus states and associated labels.

Final frontend quality commands

```
npm run lint
npm run build
```

Checkpoint: Do not continue until every item below is true.

- ☐ Register/login/logout works through the ASP.NET Core cookie.
- ☐ Project and question CRUD required for Part 1 works.
- ☐ Refresh preserves authenticated session and DB data.
- ☐ npm run lint passes.
- ☐ npm run build passes.

12. React hook review and Part 2 image plan

12.1 Hook review by actual responsibility

Hook	Concrete lesson usage	Enterprise lesson
useState	Form input and auth status	Keep state local until multiple distant components actually need it.
useEffect	Session bootstrap, data load, ResizeObserver	Effects synchronize React with external systems; they are not a general place to compute derived values.
useContext	Auth state/actions	Context is appropriate for truly cross-cutting state, not every piece of server data.
useReducer	Workspace async state	Reducers make related transitions explicit and testable.
useCallback	Auth/workspace actions	Stable function identities matter when functions cross memoized component/context boundaries.
useMemo	Filtered project list and AuthContext value	Memoize derived values where identity or computation matters; do not blanket every expression.
useRef	Canvas DOM node and request ordering	Refs hold mutable values that do not need to trigger a render.
useId	Form accessibility	Generate stable IDs for reusable component instances.
useDeferredValue	Project search/filter	Defer non-critical rendering work without delaying the input itself.

12.2 Part 2 server-image flow

- User selects an owned saved question.
- React POSTs to /api/projects/{projectId}/questions/{questionId}/generate.

13. Backend verifies project/question ownership before generating anything expensive.
14. Backend returns 200 image/png with binary image bytes.
15. Frontend calls `response.blob()`, then `createImageBitmap(blob)` or loads an object URL.
16. Canvas 2D context draws the image at full backing resolution.
17. Sharpen/noise controls dispatch reducer actions that update processing parameters.
18. An image-processing function reads `ImageData`, applies kernels/filters, and writes pixels back with `putImageData`.
19. Keep an untouched original `ImageData` or `ImageBitmap` so Reset never compounds filters.
20. Use a Web Worker if CPU-heavy filters make the UI thread stutter; keep React responsible for UI state, not per-pixel loops.

12.3 Suggested Part 2 React state

Part 2 design sketch - do not implement yet

```
type ImageState = {
  status: 'empty' | 'loading' | 'ready' | 'processing' | 'error'
  sharpen: number
  noiseReduction: number
  original: ImageBitmap | null
  error: string | null
}

// Good Part 2 candidates:
// useReducer      -> image state transitions
// useRef         -> canvas + offscreen/original buffers
// useEffect       -> redraw when the selected processed image changes
// useCallback     -> generate/reset/download actions
// useTransition   -> optional non-urgent UI state around expensive reprocessing
```

Image quality rule: Always perform processing from a preserved original or a controlled pipeline. Repeatedly sharpening an already sharpened canvas makes parameter changes non-deterministic and quickly degrades the image.

Appendix A. Final frontend file tree

Expected structure

```
frontend/
  src/
    api/
      http.ts
    auth/
      AuthContext.tsx
      useAuth.ts
    components/
      AppErrorBoundary.tsx
      AuthPanel.tsx
      ImageWorkspace.tsx
      ProjectForm.tsx
      QuestionForm.tsx
    hooks/
      useWorkspace.ts
    types/
      api.ts
    App.css
    App.tsx
    index.css
    main.tsx
  index.html
  package.json
  tsconfig.json
  vite.config.ts
```

Appendix B. Troubleshooting

Symptom	Likely cause	Fix
Vite proxy returns ECONNREFUSED	Backend is not running or port differs	Start API at https://localhost:7043 and verify

Symptom	Likely cause	Fix
		launchSettings.json.
Browser shows 401 immediately after successful login	Cookie not being sent	Confirm apiFetch uses credentials: include and requests use relative /api URLs through Vite.
Registration returns 400	Password policy not satisfied or email invalid	Inspect ProblemDetails/errors in DevTools and use the backend 12-character policy.
Project create returns 409	Same user already has normalized project name	Show the API error; choose a different name.
Canvas looks blurry later	CSS size and pixel buffer differ	Keep the devicePixelRatio resize logic before Part 2 drawing.
Selected project briefly changes to an older click	Out-of-order responses	Keep detailsRequestId useRef guard.
Effect appears to run twice in development	React StrictMode	Expected development behavior; effects must be safe and cleanup correctly.
npm run build fails on unused imports	TypeScript/ESLint strictness	Remove unused starter imports/files and follow exact file contents.
CORS error despite Vite proxy	Code called full API URL instead of /api	Use relative API URLs so the dev proxy is used.
Cookie visible to JavaScript is missing	HttpOnly cookie protection	Expected. The app should never read the auth cookie directly.

Reference baseline

- React documentation: Using TypeScript and built-in React Hooks.
- Vite documentation: Getting Started and React TypeScript template.
- Microsoft Learn: ASP.NET Core Identity backend for SPAs recommends cookies for browser applications.
- Backend Part 1 PDF from this lesson set defines the API contract used here.